

Universidade Federal de Itajubá - IESTI

2a Prova de ELT024 – Programação para Sistemas Embarcados

Prof. Maurílio Cereira Coutinho

(20 pts Questão 1) Escreva uma **função** de tratamento de interrupções. Esta função será responsável por armazenar o valor **AD** numa variável global para que o programa principal possa ler este valor posteriormente. O resultado da conversão do AD é armazenado no registro **ADRES**. A flag responsável por indicar fim de conversão é o **bit 6** do registro **PIR1**.

(25 pts Questão 2) Crie um **programa** que leia um sensor de pressão que está conectado na entrada analógica e apresente o seu valor nos 3 primeiros dígitos do display de 7 segmentos. Para evitar o flicker o loop principal deve ser executado na velocidade de 4ms. O último dígito do display de 7 segmentos deve mostrar a letra 'b'. Quando o sensor está sob a ação de uma pressão de 0 bar o conversor analógico lê 0 volts (0 no AD). Sob a pressão de 256 bar a saída fica saturada em 5volts (1024 no AD). O sensor possui uma resposta linear. Utilize as bibliotecas “display.h”, “timer.h” e “adc.h”.

(25 pts Questão 3) Um determinado motor possui no seu eixo um sensor indutivo para medição de rotação/distância percorrida. Esse sensor é digital e apresenta 5 volts na saída sempre que um dente da engrenagem passa por ele. A engrenagem possui 32 dentes. Este sensor será conectado numa placa no bit 3 da porta E do microcontrolador. Esta placa possui ainda um LCD e um botão ligado no bit 2 da porta E. Este botão serve para zerar a contagem de voltas. Faça um **programa** que conte quantas voltas o motor já deu desde a última vez que o botão foi pressionado e exiba o valor no LCD. Pode utilizar a biblioteca “lcd.h”. Cuidado com o problema de bouncing do sinal do sensor. O botão não possui problema de bouncing.

(30 pts Questão 4) Um determinado sistema realiza o controle de geração de energia de uma turbina eólica. A energia gerada deve sempre estar na mesma frequência da rede para que possa ser transmitida sem problemas. Um dos modos de se realizar esse controle é modificando a angulação das pás para controlar a velocidade de rotação da turbina. Se não houver vento suficiente o sistema deve se desconectar da rede. Se o vento for maior do que o possível de ser controlado o freio de emergência deve ser acionado.

O controle das pás é feito através de um sistema de servomotor acionado através de uma saída PWM, onde 0% representa a pá a 0 graus (para baixa velocidade de vento) e 100% representa a pá a 90 graus (para altas velocidades de vento). A rotação atual da turbina pode ser lida através da entrada analógica. Seu valor vai de 0 à 1023, onde 0 representa 0 Hz e 600 representa 60Hz. O freio de emergência é acionado através do bit 3 da porta C. O bit 6 da porta C aciona o relé que faz a conexão da turbina eólica com a rede. O ângulo ótimo da pá é dado pela fórmula: $\text{ângulo_pá} = (45 - (\text{freq_rotor} - \text{freq_rede}))$. Se o ângulo final for maior que 90 graus é porque a velocidade do vento está mais alta do que a permitida. Se o ângulo final for menor que 0 a velocidade do vento não é suficiente.

Faça um **programa** que realize o controle da turbina de modo que a rotação seja ajustada em 60Hz.

Funções disponíveis nas bibliotecas (usar de acordo com o enunciado dos problemas).

```
//lcd.h
void InicializaLCD(void);
void EnviaComando(char valor);
void EnviaDados(char valor);
//timer.h
void InicializaTimer(void);
//tempo em us
void ResetaTimer(unsigned int tempo);
void AguardaTimer(void);
//7seg.h
void Inicializa7Seg(void);
void AtualizaDisplay(void);
void MudaDigito(char valor, char posicao);
//adc.h
void InicializaAD(void);
unsigned int LeValorAD(void);
```

```
//pwm.h
void InicializaPWM(void);
void SetaFreqPWM(int freq);
void SetaPWM(int porcentagem);
```

RESPOSTAS

RESPOSTA EX 1

5 pt pra criar a variável global
5 pt pra lembrar do interrupt 1
5 pt pra testar certo o bit de flag e limpar a flag depois
5 pt pra ler o valor/armazenar na variavel

```
static int ValorADGlobal;
void ISR (void) interrupt 1 { //o nome da função não importa
    if (BitTst(PIR1,6)){ // teste para saber se a flag do AD está ligada
        ValorADGlobal = ADRES;
        BitClr(PIR1,6);
    }
}
```

RESPOSTA EX 2

5 pt inicialização dos 3 sistemas.
5 pt pra fazer o loop temporizado corretamente
10 pt por ler o valor da pressão e converter corretamente
5 pt por separar os algarismos corretamente
(se usar um loop for pra contar tempo tira uns 5 pontos)

```
#include "basico.h"
#include "config.h"
#include "disp7seg.h"
#include "adc.h"
void main (void){
    int pressao;
    InicializaDisplays();
    InicializaAD();
    InicializaTimer();
    for(;;){
        ResetaTimer(4000); //4ms
        pressao = LeValorAD();
        pressao /= 4;
        MudaDisplay((pressao/100)%10, 3);
        MudaDisplay((pressao/10)%10, 2);
        MudaDisplay((pressao/1)%10, 1);
        MudaDisplay(11, 0); //11 = b em hexadecimal
        AguardaTimer(); //Tempo pra evitar flicker
    }
}
```

RESPOSTA EX 3

5 pt inicialização do LCD e dos botões como saída (TRIS)
5 pt pra zerar a contagem com o botão 2
10 pt pro debounce do botão 3 (se tentar fazer um debounce, já conta alguns pontos)
5 pt por separar os algarismos e imprimir no LCD

```

#include "basico.h"
#include "config.h"
#include "lcd.h"
void main (void) {
    unsigned int cont = 0;
    unsigned char tempoDebounce = 0;
    unsigned char sensorAntigo;
    unsigned char sensorNovo;
    BitSet(TRISE, 2); //botão
    BitSet(TRISE, 3); //sensor
    InicializaLCD();
    for(;;){
        if(BitTst(PORTE, 2)){
            cont = 0;
        }
        if(BitTst(PORTE, 3) != 0){ //sensor ligado, mas pode ser ruído
            sensorNovo = 1;
            if(sensorAntigo == 0){ //o estado anterior era desligado
                tempoDebounce = 10;
            }
        }
        if(BitTst(PORTE, 3) == 0){ //sensor desligado, mas pode ser ruído
            sensorNovo = 0;
            if(sensorAntigo == 1){ //o estado anterior era ligado
                tempoDebounce = 10;
            }
        }
        if (sensorAntigo != sensorNovo){
            tempoDebounce--;
        }
        if(tempoDebounce == 0){
            if(sensorAntigo == 0){
                cont++; //acabou de acontecer transição de 0 para 1
            }
            sensorAntigo = sensorNovo;
        }
        EnviaComando(0x80);
        //divide por 32, quantidade de dentes
        EnviaDados((sensor/32000)%10);
        EnviaDados((sensor/3200)%10);
        EnviaDados((sensor/320)%10);
        EnviaDados((sensor/32)%10);
    }
}

```

RESPOSTA EX 4

5 pt inicialização do LCD e dos botões como saída (TRIS)

10 pt pra ler o valor no ad calcular corretamente o ângulo = $(45 - (\text{LeValorAD}()/10.0 - 60))$;

10 pt pra garantir os limites (>90 e <0) e executar as atividades de freio/desconexão

5 pt por ajustar o PWM corretamente de acordo com o valor do ângulo (apenas entre 0 e 90)

```

#include "basico.h"
#include "config.h"
#include "pwm.h"
#include "adc.h"
void main (void){
    int ang;
    BitClr(TRISC, 3); //config relé como saída
    BitClr(TRISC, 6); //config freio como saída
    InicializaPWM();
    InicializaAD();
    SetaFreqPWM(10000); //é importante configurar, o valor não é crítico
    for(;;){
        ang = (45 - (LeValorAD()/10.0 - 60));

        if(ang >=90){ //vento muito forte
            BitSet(PORTC,6); //liga freio
        }else{
            BitClr(PORTC,6); //desliga freio
        }

        if(ang <=0){ //vento muito fraco
            BitClr(PORTC,3); //desconecta à rede
        }else{
            BitClr(PORTC,6); //conecta à rede
        }
        if(ang>=0 && ang<=90){
            SetaPWM(ang * 100 / 90);
        }
    }
}

```